



SOFTWARE REQUIREMENTS SPECIFICATION DOCUMENT

Introducing Our Idea:

CODE Wave *Flow with the Next Wave of Code.*

PREPARED For

Application Development Course Fall 2025-2026

Doctor Mohamad Ghandor

PREPARED By

Haydar Mokdad

Fatima Masnour

Nour Hammoud

Table of Contents

1. Executive Summary
 2. Project Overview
 3. Architecture & Design Patterns
 4. Common Requirements Implementation
 5. Personal Requirements
 6. Functional Requirements
 7. Technical Specifications
 8. API Documentation
 9. Database Schema
 10. Security & Authentication
 11. Testing & Quality Assurance
 12. Deployment & DevOps
 13. Appendices
-

1. Executive Summary

1.1 Project Description

CodeWave is a comprehensive, interactive online learning platform designed to teach programming languages (Python and Java) through structured courses, hands-on coding exercises, quizzes, and real-time progress tracking. The platform serves both learners seeking to master programming skills and administrators managing the learning content and user base.

1.2 Key Objectives

- Provide an interactive, engaging learning experience for programming education
- Implement clean architecture with separation of concerns
- Ensure scalability, maintainability, and extensibility
- Provide comprehensive API for external integrations
- Implement real-time features for enhanced user experience
- Maintain high code quality through testing and validation

1.3 Technology Stack

- **Framework:** ASP.NET Core 9.0 MVC
- **Database:** MySQL 8.0 (with SQL Server support)
- **ORM:** Entity Framework Core 9.0
- **Authentication:** ASP.NET Core Identity with OAuth (Google, GitHub)

- **Frontend:** Razor Views, Tailwind CSS, JavaScript
 - **Code Execution:** Docker-based sandbox environments
 - **Logging:** Serilog
 - **Real-time:** SignalR
 - **Mapping:** AutoMapper
 - **Testing:** xUnit, Moq
-

2. Project Overview

2.1 System Purpose

CodeWave provides a complete Learning Management System (LMS) for programming education, featuring:

- Interactive code editor with real-time execution
- Structured learning paths (Python and Java)
- Assessment and quiz systems
- Progress tracking and analytics
- Career tools (job offers, CV generation)
- Admin dashboard for content management

2.2 User Roles

2.2.1 Learners

- **Access Level:** Standard user account
- **Capabilities:**
 - Browse and enroll in courses
 - Access lessons and exercises
 - Execute code in real-time
 - Take quizzes and assessments
 - Track learning progress
 - Generate CV based on achievements
 - Apply for job offers
 - Use AI helper in focus mode

2.2.2 Administrators

- **Access Level:** Admin account with full privileges
- **Capabilities:**
 - Manage users (create, edit, delete, toggle admin status)
 - Manage courses (CRUD operations)

- Manage job offers
- View analytics and reports
- Access admin dashboard with real-time metrics
- Manage platform content

2.3 Core Features

1. User Management System

- Registration (email/password, OAuth)
- Authentication and authorization
- Profile management
- Role-based access control

2. Learning Management

- Course browsing and enrollment
- Interactive lessons with rich content
- Coding exercises with test cases
- Real-time code execution (Java, Python)
- Progress tracking

3. Assessment System

- Initial skill assessment
- Learning path assignment
- Quizzes based on lessons
- Performance analytics

4. Admin Dashboard

- User engagement metrics (Chart.js)
- Skill distribution analytics
- Course enrollment statistics
- Real-time data visualization

5. Career Tools

- Job offer listings with search
- Auto-generated CV (PDF)
- Project portfolio

6. Focus Mode

- Distraction-free learning environment
 - Integrated code editor
 - AI helper chatbot (Google Gemini API)
 - Popup chatbot interface
-

3. Architecture & Design Patterns

3.1 Four-Layer Architecture

The application follows Clean Architecture principles with four distinct layers:

3.1.1 Domain Layer (`CodeWave.Domain`)

- **Purpose:** Core business entities and domain logic
- **Components:**
 - Entity classes (22 entities)
 - Domain models
 - Business rules
- **Dependencies:** None (pure domain logic)
- **Key Entities:**
 - `ApplicationUser`, `Course`, `Lesson`, `CodingExercise`
 - `Quiz`, `QuizQuestion`, `Assessment`, `JobOffer`
 - `CV`, `Project`, `UserCourse`, `LessonCompletion`

3.1.2 Application Layer (`CodeWave.Application`)

- **Purpose:** Application logic, interfaces, DTOs, ViewModels
- **Components:**
 - Service interfaces (18 interfaces)
 - DTOs (6 DTO classes)
 - ViewModels (3 ViewModels)
 - Mapping profiles (AutoMapper)
- **Dependencies:** Domain layer only
- **Key Interfaces:**
 - `IUserService`, `ICourseRepository`, `ILessonRepository`
 - `IProgressService`, `IQuizService`, `ICodeService`
 - `IUnitOfWork`, `IAssessmentService`

3.1.3 Infrastructure Layer (`CodeWave.Infrastructure`)

- **Purpose:** Data access, external services, implementations
- **Components:**

- Repositories (9 repository implementations)
- Services (13 service implementations)
- Data context (Entity Framework)
- Migrations (16 migrations)
- Seed data (6 seed classes)
- **Dependencies:** Application and Domain layers
- **Key Components:**
 - `ApplicationDbContext`
 - Repository implementations
 - Service implementations
 - Code execution services

3.1.4 Web Layer (`CodeWave.Web`)

- **Purpose:** User interface, controllers, views, API endpoints
- **Components:**
 - Controllers (19 controllers)
 - Views (52 Razor views)
 - Hubs (SignalR)
 - Background services
 - Static files (CSS, JS)
- **Dependencies:** All other layers
- **Key Components:**
 - MVC controllers
 - API controllers
 - SignalR hubs
 - Background services

3.2 Repository Pattern

Implementation: All data access operations are abstracted through repository interfaces.

3.2.1 Repository Interfaces (Application Layer)

- `ICourseRepository`
- `ILessonRepository`
- `IExerciseRepository`
- `IExerciseSubmissionRepository`
- `ILessonCompletionRepository`
- `IQuizRepository`

- IProjectRepository

- IUserCourseRepository

3.2.2 Repository Implementations (Infrastructure Layer)

- All repositories implement their respective interfaces
- Use Entity Framework Core for data access
- Provide CRUD operations and custom queries
- Support soft delete pattern

Example:

```
public class CourseRepository : ICourseRepository
{
    private readonly ApplicationDbContext _context;

    // Implementation with EF Core
}
```

3.3 Unit of Work Pattern

Implementation: Centralized transaction management and database operations.

3.3.1 Interface

```
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync();
}
```

3.3.2 Implementation

```
public class UnitOfWork : IUnitOfWork
{
    private readonly ApplicationDbContext _context;
```

```
public Task<int> SaveChangesAsync() => _context.SaveChangesAsync();  
  
}
```

3.3.3 Usage in Services

All services use `IUnitOfWork` for transaction management:

- `CodeService`
- `LearningPathService`
- `CVService`
- `JobApplicationService`

Benefits:

- Ensures data consistency
 - Simplifies transaction management
 - Reduces database round trips
-

4. Common Requirements Implementation

4.1 Four Layers Implemented

Status: Fully Implemented

Evidence:

- Domain Layer: `CodeWave.Domain` project with 22 entities
- Application Layer: `CodeWave.Application` with interfaces, DTOs, ViewModels
- Infrastructure Layer: `CodeWave.Infrastructure` with repositories and services
- Web Layer: `CodeWave.Web` with controllers, views, and API endpoints

Documentation:

- Clear separation of concerns
- Dependency flow: Web → Infrastructure → Application → Domain
- No circular dependencies

4.2 Repository Pattern

Status: Fully Implemented

Evidence:

- 8 repository interfaces in Application layer
- 9 repository implementations in Infrastructure layer
- All data access abstracted through repositories
- Services depend on interfaces, not implementations

Key Repositories:

- `CourseRepository`, `LessonRepository`, `ExerciseRepository`
- `QuizRepository`, `ProjectRepository`, `UserCourseRepository`

4.3 Unit of Work Applied in Managers (1 point)

Status: Fully Implemented

Evidence:

- `IUnitOfWork` interface defined
- `UnitOfWork` implementation in Infrastructure
- Used in all service classes (managers)
- Centralized transaction management

Services Using UnitOfWork:

- `CodeService`, `LearningPathService`, `CVService`, `JobApplicationService`

4.4 Authentication (2 points)

Status: Fully Implemented

Implementation Details:

4.4.1 ASP.NET Core Identity

- User registration with email/password
- Password hashing and validation
- Session management
- Cookie-based authentication

4.4.2 OAuth Integration

- **Google OAuth:** `AspNet.Security.OAuth.Google`
- **GitHub OAuth:** `AspNet.Security.OAuth.GitHub`

- External authentication providers
- Automatic account creation for OAuth users

4.4.3 Authentication Flow

1. User selects authentication method
2. For OAuth: Redirect to provider
3. Provider callback with user info
4. Create or link account
5. Generate session cookie
6. Redirect based on role (Admin → Admin Dashboard, User → Home)

Files:

- `CodeWave.Web/Controllers/UserController.cs`
- `CodeWave.Web/Program.cs` (OAuth configuration)
- `CodeWave.Web/Models/AuthViewModels.cs`

4.5 Multiple Roles with Multiple Privileges (3 points)

Status: Fully Implemented

Implementation:

4.5.1 Role System

- **Regular User:** Standard learning features
- **Admin:** Full system access

4.5.2 Authorization Policies

```
builder.Services.AddAuthorization(options =>
```

```
{
```

```
options.AddPolicy("AdminOnly", policy =>
```

```
policy.RequireAssertion(context =>
```

```
context.User.Identity.IsAuthenticated &&
```

```
context.User.HasClaim("IsAdmin", "true")));
```

```
});
```

4.5.3 Role-Based Features

Regular Users:

- Course enrollment and learning
- Code execution
- Quiz taking
- Progress tracking
- CV generation
- Job application

Administrators:

- All user features PLUS:
- User management (CRUD)
- Course management (CRUD)
- Job offer management
- Admin dashboard access
- Analytics and reports

4.5.4 Claims-Based Authorization

- `IsAdmin` claim for admin users
- Custom `UserClaimsPrincipalFactory` for claim injection
- Policy-based authorization on controllers

Files:

- `CodeWave.Web/Claims/UserClaimsPrincipalFactory.cs`
- `CodeWave.Web/Controllers/AdminController.cs` (uses `[Authorize(Policy = "AdminOnly")]`)

4.6 Logging (Serilog) (3 points)

Status: Fully Implemented

Implementation:

4.6.1 Configuration

```
Log.Logger = new LoggerConfiguration()
```

```
.WriteTo.Console()
```

```
.WriteTo.File("logs/codewave-.txt", rollingInterval: RollingInterval.Day)
```

```
.CreateLogger();
```

```
builder.Host.UseSerilog();
```

4.6.2 Logging Features

- **Console Sink:** Real-time logging to console
- **File Sink:** Daily rolling log files
- **Structured Logging:** JSON-formatted logs
- **Log Levels:** Information, Warning, Error, Debug

4.6.3 Usage Throughout Application

- API controllers log requests and errors
- Services log business logic events
- Background services log execution
- Exception handling with detailed logging

Log File Location: `logs/codewave-{date}.txt`

Example Usage:

```
_logger.LogInformation("User {UserId} enrolled in course {CourseId}", userId, courseId);
```

```
_logger.LogError(ex, "Error processing payment for user {UserId}", userId);
```

Files:

- `CodeWave.Web/Program.cs` (Serilog configuration)
- All controllers and services use `ILogger<T>`

4.7 Expose API to be Used by Other Possible Party (3 points)

Status: Fully Implemented

Implementation:

4.7.1 API Controllers

- **Location:** `CodeWave.Web/Controllers/Api/`
- **Base Route:** `/api/v1/`

4.7.2 Available API Endpoints

Courses API (/api/v1/CoursesApi):

- GET /api/v1/CoursesApi - Get all courses (public)
- GET /api/v1/CoursesApi/{id} - Get course by ID (public)
- GET /api/v1/CoursesApi/my-courses - Get user's enrolled courses (authenticated)

Code Execution API (/api/code):

- POST /api/code/run - Execute code (authenticated)
- Returns: output, test results, pass/fail status

Code Runner API (/api/CodeRunner):

- POST /api/CodeRunner/run-java - Execute Java code

Progress API (/api/progress):

- POST /api/progress/completeLesson - Mark lesson as complete

4.7.3 API Features

- RESTful design
- JSON responses
- Error handling with appropriate HTTP status codes
- Authentication support (JWT-ready)
- XML documentation comments
- DTOs for request/response

4.7.4 API Documentation

- XML comments for Swagger generation
- Proper HTTP status codes (200, 400, 401, 404, 500)
- Error response models

Example Response:

```
{  
  
  "id": "guid",  
  
  "title": "Python Basics",
```

```
"description": "...",  
  
"difficulty": "Beginner",  
  
"language": "Python",  
  
"lessonCount": 10,  
  
"createdAt": "2025-01-01T00:00:00Z"  
  
}
```

Files:

- `CodeWave.Web/Controllers/Api/CoursesApiController.cs`
- `CodeWave.Web/Controllers/CodeController.cs`
- `CodeWave.Web/Controllers/CodeRunnerController.cs`

4.8 Background Service (2 points)

Status: Fully Implemented

Implementation:

4.8.1 CleanupBackgroundService

- **Type:** `IHostedService`
- **Location:** `CodeWave.Web/Services/CleanupBackgroundService.cs`
- **Execution:** Runs daily (24-hour interval)

4.8.2 Features

1. **Temp File Cleanup:** Removes temporary files older than 7 days
2. **Soft-Delete Cleanup:** Permanently removes soft-deleted records older than 90 days
3. **Logging:** Integrated with Serilog for operation tracking
4. **Error Handling:** Graceful error handling with logging

4.8.3 Registration

```
builder.Services.AddHostedService<CleanupBackgroundService>();
```

4.8.4 Benefits

- Automatic maintenance
- Database optimization

- Storage management
- No manual intervention required

Files:

- `CodeWave.Web/Services/CleanupBackgroundService.cs`
- `CodeWave.Web/Program.cs` (registration)

4.9 Unit Test (2 points)

Status: Fully Implemented

Implementation:

4.9.1 Test Project

- **Project:** `CodeWave.Tests`
- **Framework:** xUnit 2.9.2
- **Mocking:** Moq 4.20.72
- **Database:** EF Core InMemory 9.0.0

4.9.2 Test Coverage

Service Tests:

- `CodeServiceTests.cs` - Code execution service tests
- Tests for valid code execution
- Tests for empty code handling

Repository Tests:

- `UnitOfWorkTests.cs` - Unit of Work pattern tests
- Database transaction tests
- Save changes verification

4.9.3 Test Structure

[Fact]

```
public async Task SaveChangesAsync_ShouldSaveChangesToDatabase()

{

    // Arrange
```

```
// Act

// Assert

}
```

4.9.4 Running Tests

dotnet test CodeWave.Tests/CodeWave.Tests.csproj

Files:

- `CodeWave.Tests/CodeWave.Tests.csproj`
- `CodeWave.Tests/Services/CodeServiceTests.cs`
- `CodeWave.Tests/Repositories/UnitOfWorkTests.cs`

4.10 Client Validation (2 points)

Status: Fully Implemented

Implementation:

4.10.1 ASP.NET Core Built-in Validation

- jQuery Validation Unobtrusive
- Data annotations on view models
- Real-time validation feedback

4.10.2 Enhanced Client Validation

File: `CodeWave.Web/wwwroot/js/validation-enhancements.js`

Features:

1. **Email Validation:** Real-time email format checking
2. **Password Strength Indicator:** Visual strength meter
 - Very Weak, Weak, Fair, Good, Strong
 - Color-coded progress bar
3. **Password Confirmation:** Real-time matching validation
4. **Visual Feedback:** Error messages with styling
5. **Field Highlighting:** Invalid fields highlighted in red

4.10.3 Validation Rules

- Email: Must match email regex pattern
- Password: Minimum 6 characters, strength indicators

- Password Confirmation: Must match password
- Required fields: Visual indication

Files:

- `CodeWave.Web/wwwroot/js/validation-enhancements.js`
- `CodeWave.Web/Views/Shared/_ValidationScriptsPartial.cshtml`
- All form views include validation scripts

4.11 Server Validation (2 points)

Status: Fully Implemented

Implementation:

4.11.1 Data Annotations

All view models use data annotations:

```
public class RegisterViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [DataType(DataType.Password)]
    [StringLength(100, MinimumLength = 6)]
    public string Password { get; set; }

    [Required]
    [Compare("Password")]
    public string ConfirmPassword { get; set; }
}
```

4.11.2 ModelState Validation

All controllers validate ModelState:

[HttpPost]

```
public async Task<IActionResult> Register(RegisterViewModel model)

{

    if (!ModelState.IsValid)

    {

        return View(model);

    }

    // Process registration

}
```

4.11.3 Custom Validation

- Password strength requirements
- Email uniqueness validation
- Business rule validation in services

Files:

- `CodeWave.Web/Models/AuthViewModels.cs`
- All controller action methods

4.12 Queries Optimized (2 points)

Status: Fully Implemented

Implementation:

4.12.1 EF Core Query Optimization

- **Eager Loading:** Use `.Include()` for related entities
- **Projection:** Select only needed fields
- **Filtering:** Apply filters before materialization

- **Indexing:** Database indexes on foreign keys and search fields

4.12.2 Optimized Queries Examples

Example 1: Course with Lessons

```
var course = await _context.Courses  
  
    .Include(c => c.Lessons)  
  
    .FirstOrDefaultAsync(c => c.Id == id && !c.IsDeleted);
```

Example 2: User Progress with Projection

```
var progress = await _context.LessonCompletions  
  
    .Where(lc => lc.UserId == userId)  
  
    .Select(lc => new { lc.LessonId, lc.CompletedAt })  
  
    .ToListAsync();
```

4.12.3 Database Optimization

- Soft delete pattern (avoids physical deletes)
- Proper indexing on frequently queried fields
- Connection pooling
- Query caching where appropriate

4.13 Use DTOs, VMs and Mapping (2 points)

Status: Fully Implemented

Implementation:

4.13.1 DTOs (Data Transfer Objects)

Location: [CodeWave.Application/DTOs/](#)

DTOs:

- [LearningPathDtos.cs](#) - Course, Lesson, Exercise DTOs
- [ProgressDtos.cs](#) - Progress tracking DTOs
- [QuizDtos.cs](#) - Quiz-related DTOs
- [RunCodeRequestDto.cs](#) - Code execution DTO

- `SubmitAssessmentDto.cs` - Assessment DTO
- `TakeAssessmentDto.cs` - Assessment taking DTO

4.13.2 ViewModels

Location: `CodeWave.Application/ViewModels/`

ViewModels:

- `FocusModeViewModel.cs` - Focus mode view data
- `JavaLearningPathViewModel.cs` - Java path view data
- `LearningPathViewModel.cs` - General learning path data

Web ViewModels:

- `CodeWave.Web/Models/AuthViewModels.cs` - Authentication view models

4.13.3 AutoMapper

Configuration: `CodeWave.Application/Mappings/MappingProfile.cs`

Features:

- Entity to DTO mapping
- DTO to Entity mapping
- ViewModel mapping
- Registered in DI container

Registration:

```
builder.Services.AddAutoMapper(typeof(MappingProfile));
```

Usage:

```
var courseDto = _mapper.Map<CourseDto>(course);
```

Files:

- `CodeWave.Application/Mappings/MappingProfile.cs`
- `CodeWave.Application/DTOs/` (6 files)
- `CodeWave.Application/ViewModels/` (3 files)

4.14 Dashboard (for each user show relevant data) (3 points)

Status: Fully Implemented

Implementation:

4.14.1 User Dashboard

Controller: `HomeController` **View:** `CodeWave.Web/Views/Home/Index.cshtml`

Features:

- User progress statistics
- Enrolled courses
- Recent lesson completions
- Exercise submissions
- Quiz attempts
- Acquired skills
- Learning path information

4.14.2 Admin Dashboard

Controller: `AdminController` **View:** `CodeWave.Web/Views/Admin/Index.cshtml`

Features:

1. **User Engagement Chart** (Chart.js)
 - Daily active users
 - Lesson completions over time
 - Interactive line chart
2. **Skill Distribution Chart** (Chart.js)
 - Beginner, Intermediate, Advanced percentages
 - Interactive bar chart
3. **Courses List**
 - All courses with enrollment counts
 - Course details and statistics
4. **Real-time Data**
 - Dynamic data from database
 - Updated on page load

4.14.3 Dashboard Data Sources

- User-specific: Filtered by `UserId`
- Admin: Aggregated across all users
- Real-time: Queries database on each request

Files:

- `CodeWave.Web/Controllers/HomeController.cs`
- `CodeWave.Web/Controllers/AdminController.cs`
- `CodeWave.Web/Views/Home/Index.cshtml`
- `CodeWave.Web/Views/Admin/Index.cshtml`

4.15 Contents Pages (Variance of Page contents, number of Pages, Quality of Page Contents code) (4 points)

Status: Fully Implemented

Implementation:

4.15.1 Page Count

Total Views: 52 Razor views

Page Categories:

1. **Authentication Pages** (4 pages)
 - Login, SignUp, ForgotPassword, ResetPassword
2. **Learning Pages** (15+ pages)
 - Welcome, Home Dashboard
 - Course listings, Course details
 - Lesson views, Focus mode
 - Python/Java learning paths
3. **Exercise Pages** (5+ pages)
 - Code editor, Exercise views
 - Test results, Submission history
4. **Quiz Pages** (5+ pages)

- Quiz list, Quiz taking
- Quiz results, Question views

5. **Admin Pages** (10+ pages)

- Admin dashboard
- User management (CRUD)
- Course management (CRUD)
- Job offer management

6. **Career Pages** (5+ pages)

- Job offers list, Job details
- CV builder, CV view
- Projects

7. **Settings Pages** (3+ pages)

- Profile settings
- Password change
- Account management

4.15.2 Content Quality

- **Rich Content:** Markdown support, code highlighting
- **Interactive Elements:** Code editor, charts, forms
- **Responsive Design:** Mobile-friendly layouts
- **Accessibility:** Semantic HTML, ARIA labels
- **SEO:** Meta tags, structured data

4.15.3 Code Quality

- **Separation of Concerns:** Views only for presentation
- **Reusable Components:** Partial views, layouts
- **Clean Code:** Well-structured, commented
- **Consistent Styling:** Tailwind CSS throughout

Files: All views in `CodeWave.Web/Views/`

4.16 Theme (2 points)

Status: Fully Implemented

Implementation:

4.16.1 Dark/Light Theme

- **Framework:** Tailwind CSS with dark mode
- **Toggle:** Theme switcher in navigation
- **Persistence:** LocalStorage for theme preference
- **Automatic:** Respects system preference

4.16.2 Theme Implementation

// Theme toggle functionality

```
function toggleTheme() {  
  
    document.documentElement.classList.toggle('dark');  
  
    localStorage.setItem('theme', ...);  
  
}
```

4.16.3 Theme Features

- Smooth transitions
- Consistent color scheme
- Accessible contrast ratios
- Custom color palette (purple primary)

Files: All views use Tailwind dark mode classes

4.17 Responsive Theme (1 point)

Status: Fully Implemented

Implementation:

4.17.1 Responsive Design

- **Framework:** Tailwind CSS responsive utilities
- **Breakpoints:** sm, md, lg, xl, 2xl
- **Mobile-First:** Designed for mobile, enhanced for desktop

4.17.2 Responsive Features

- **Navigation:** Collapsible mobile menu
- **Grid Layouts:** Responsive grid systems
- **Forms:** Stack on mobile, side-by-side on desktop
- **Tables:** Scrollable on mobile

- **Charts:** Responsive Chart.js configurations

4.17.3 Testing

- Tested on multiple screen sizes
- Mobile devices (320px+)
- Tablets (768px+)
- Desktops (1024px+)

Files: All views use Tailwind responsive classes

4.18 Consume External API (Extra) (2 points)

Status: Fully Implemented

Implementation:

4.18.1 Google Gemini API

Purpose: AI helper chatbot in focus mode

Integration:

- **Endpoint:**
<https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent>
- **Authentication:** API key ([AIzaSyBUW0eYv9520rDu4zHySRo0l2lCYvwhs_U](#))
- **HttpClient:** Uses [IHttpClientFactory](#)

Features:

- Context-aware responses
- Code explanation
- Learning assistance
- Markdown response rendering

Implementation:

```
var response = await _httpClient.PostAsync(apiUrl, content);
```

```
var responseContent = await response.Content.ReadAsStringAsync();
```

Files:

- `CodeWave.Web/Controllers/LearningPathController.cs` (GenerateAIResponse method)
- `CodeWave.Web/Views/LearningPath/FocusModeLesson.cshtml` (Chatbot UI)

4.18.2 OAuth APIs

- Google OAuth API
- GitHub OAuth API

4.19 SignalR (Extra) (2 points)

Status: Fully Implemented

Implementation:

4.19.1 SignalR Hub

Hub: `NotificationHub` **Location:** `CodeWave.Web/Hubs/NotificationHub.cs` **Route:** `/notificationHub`

4.19.2 Features

1. Real-time Notifications

- User-specific notifications
- Progress updates
- System announcements

2. Progress Broadcasting

- Course progress updates
- Lesson completion notifications
- Achievement notifications

3. User Groups

- Users added to groups by userId
- Targeted messaging

4.19.3 Client Integration

File: `CodeWave.Web/wwwroot/js/signalr-client.js`

Features:

- Automatic connection

- Reconnection handling
- Notification display
- Progress update handling

Usage:

```
connection.on("ReceiveNotification", function (notification) {  
  
    showNotification(notification.message, notification.type);  
  
});
```

4.19.4 Registration

```
builder.Services.AddSignalR();
```

```
app.MapHub<NotificationHub>("/notificationHub");
```

Files:

- `CodeWave.Web/Hubs/NotificationHub.cs`
 - `CodeWave.Web/wwwroot/js/signalr-client.js`
 - `CodeWave.Web/Program.cs` (registration)
-

5. Personal Requirements

5.1 Updated DevOps Items (2 points)

Status: Implemented

DevOps Components:

- **Build Scripts:** PowerShell scripts for building
- **Migration Scripts:** Database migration scripts
- **Run Scripts:** Application startup scripts
- **Docker Support:** Docker-based code execution

Files:

- `build-safe.ps1`
- `run-app.ps1`

- `run-cv-migration.ps1`
 - Various migration scripts
-

6. Functional Requirements

6.1 User Management

6.1.1 User Registration

- **FR-1.1:** User can register with email and password
- **FR-1.2:** User can register with Google OAuth
- **FR-1.3:** User can register with GitHub OAuth
- **FR-1.4:** System validates email format
- **FR-1.5:** System validates password strength
- **FR-1.6:** System creates user account and logs in user

6.1.2 User Login

- **FR-2.1:** User can login with email and password
- **FR-2.2:** User can login with Google
- **FR-2.3:** User can login with GitHub
- **FR-2.4:** System validates credentials
- **FR-2.5:** System creates session
- **FR-2.6:** Admin users redirected to Admin Dashboard
- **FR-2.7:** Regular users redirected to Home Dashboard

6.2 Learning Management

6.2.1 Course Browsing

- **FR-5.1:** User can browse available courses
- **FR-5.2:** User can view course details
- **FR-5.3:** User can see course difficulty and language
- **FR-5.4:** User can see lesson count

6.2.2 Course Enrollment

- **FR-6.1:** User can enroll in courses
- **FR-6.2:** System tracks enrollment
- **FR-6.3:** User can view enrolled courses

6.2.3 Lesson Access

- **FR-7.1:** User can access lessons in enrolled courses
- **FR-7.2:** User can view lesson content
- **FR-7.3:** User can navigate between lessons
- **FR-7.4:** User can mark lessons as complete
- **FR-7.5:** System tracks lesson completion

6.3 Code Execution

6.3.1 Code Editor

- **FR-8.1:** User can write code in integrated editor
- **FR-8.2:** Editor provides syntax highlighting
- **FR-8.3:** Editor auto-saves code
- **FR-8.4:** User can select exercises

6.3.2 Code Execution

- **FR-9.1:** User can execute code
- **FR-9.2:** System detects programming language
- **FR-9.3:** System executes code in Docker container
- **FR-9.4:** System displays output
- **FR-9.5:** System handles errors
- **FR-9.6:** System enforces timeout
- **FR-9.7:** System logs execution

6.3.3 Exercise Submission

- **FR-10.1:** User can submit code for exercises
- **FR-10.2:** System runs test cases
- **FR-10.3:** System compares outputs
- **FR-10.4:** System displays test results
- **FR-10.5:** System marks exercise as complete if all tests pass
- **FR-10.6:** System saves submission

6.4 Assessment System

6.4.1 Initial Assessment

- **FR-4.1:** New users complete assessment
- **FR-4.2:** System presents questions
- **FR-4.3:** User answers questions
- **FR-4.4:** System evaluates answers
- **FR-4.5:** System assigns learning path (Python/Java)

- **FR-4.6:** System updates user profile

6.4.2 Quizzes

- **FR-13.1:** User can view available quizzes
- **FR-13.2:** User can take quizzes
- **FR-13.3:** System tracks time limits
- **FR-14.1:** User answers questions
- **FR-14.2:** System validates answers
- **FR-14.3:** System calculates score
- **FR-15.1:** System displays results
- **FR-15.2:** System saves attempt

6.5 Focus Mode

6.5.1 Focus Mode Features

- **FR-16.1:** User can enter focus mode
- **FR-16.2:** Distraction-free interface
- **FR-16.3:** Integrated code editor
- **FR-16.4:** Lesson content display
- **FR-16.5:** Exercise selector
- **FR-16.6:** Navigation controls

6.5.2 AI Helper

- **FR-17.1:** User can access AI helper
- **FR-17.2:** User can ask questions
- **FR-17.3:** AI provides context-aware responses
- **FR-17.4:** Responses rendered in markdown

6.6 Admin Functions

6.6.1 Admin Dashboard

- **FR-18.1:** Admin can access dashboard
- **FR-18.2:** Dashboard shows user engagement (Chart.js)
- **FR-18.3:** Dashboard shows skill distribution (Chart.js)
- **FR-18.4:** Dashboard shows course enrollments
- **FR-18.5:** Data is dynamic and real-time

6.6.2 User Management

- **FR-19.1:** Admin can view all users
- **FR-19.2:** Admin can create users
- **FR-19.3:** Admin can edit users

- **FR-19.4:** Admin can delete users
- **FR-19.5:** Admin can toggle admin status
- **FR-19.6:** Admin can search users
- **FR-19.7:** Admin can filter users
- **FR-19.8:** Admin can view user details

6.6.3 Course Management

- **FR-20.1:** Admin can create courses
- **FR-20.2:** Admin can edit courses
- **FR-20.3:** Admin can delete courses (soft delete)
- **FR-20.4:** Admin can manage lessons
- **FR-20.5:** Admin can manage exercises

6.6.4 Job Offer Management

- **FR-21.1:** Admin can create job offers
- **FR-21.2:** Admin can edit job offers
- **FR-21.3:** Admin can delete job offers
- **FR-21.4:** Admin can view applications
- **FR-21.5:** Admin can manage job details

6.7 Career Tools

6.7.1 Job Offers

- **FR-23.1:** User can browse job offers
- **FR-23.2:** User can search job offers
- **FR-23.3:** User can view job details
- **FR-23.4:** User can apply for jobs

6.7.2 CV Generation

- **FR-24.1:** User can build CV
 - **FR-24.2:** User can add projects
 - **FR-24.3:** System generates PDF CV
 - **FR-24.4:** CV includes achievements
-

7. Technical Specifications

7.1 Technology Stack

7.1.1 Backend

- **Framework:** ASP.NET Core 9.0 MVC
- **Language:** C# 12.0
- **ORM:** Entity Framework Core 9.0
- **Database:** MySQL 8.0 (primary), SQL Server (optional)
- **Authentication:** ASP.NET Core Identity
- **OAuth:** Google, GitHub

7.1.2 Frontend

- **Views:** Razor Views (.cshtml)
- **Styling:** Tailwind CSS
- **JavaScript:** Vanilla JS, jQuery
- **Charts:** Chart.js
- **Code Editor:** Monaco Editor (via CDN)

7.1.3 Infrastructure

- **Code Execution:** Docker containers
- **Logging:** Serilog
- **Mapping:** AutoMapper
- **Real-time:** SignalR
- **PDF Generation:** QuestPDF

7.1.4 Testing

- **Framework:** xUnit
- **Mocking:** Moq
- **Database:** EF Core InMemory

7.2 Project Structure

CodeWave/

- |— CodeWave.Domain/ # Domain layer (entities)
- |— CodeWave.Application/ # Application layer (interfaces, DTOs, ViewModels)
- |— CodeWave.Infrastructure/ # Infrastructure layer (repositories, services)

|— CodeWave.Web/ # Web layer (controllers, views, API)

|— CodeWave.Tests/ # Unit tests

└— CodeWave.Docker/ # Docker configurations

7.3 Database Schema

7.3.1 Core Entities

- `ApplicationUser` - User accounts
- `Course` - Learning courses
- `Lesson` - Course lessons
- `CodingExercise` - Programming exercises
- `ExerciseTestCase` - Test cases
- `ExerciseSubmission` - User submissions
- `LessonCompletion` - Lesson completion tracking
- `Quiz` - Quizzes
- `QuizQuestion` - Quiz questions
- `QuizAnswerOption` - Answer options
- `UserQuizAttempt` - Quiz attempts
- `UserQuizAnswer` - User answers
- `Assessment` - Initial assessments
- `UserAssessment` - User assessment responses
- `JobOffer` - Job listings
- `JobApplication` - Job applications
- `UserCourse` - Course enrollments
- `CV` - User CVs
- `Project` - User projects

7.3.2 Relationships

- User → Courses (Many-to-Many via UserCourse)
- Course → Lessons (One-to-Many)
- Lesson → Exercises (One-to-Many)
- Exercise → TestCases (One-to-Many)
- User → Submissions (One-to-Many)
- User → LessonCompletions (One-to-Many)
- User → QuizAttempts (One-to-Many)

7.4 API Endpoints

7.4.1 Public APIs

- `GET /api/v1/CoursesApi` - List all courses
- `GET /api/v1/CoursesApi/{id}` - Get course details

7.4.2 Authenticated APIs

- `GET /api/v1/CoursesApi/my-courses` - User's enrolled courses
- `POST /api/code/run` - Execute code
- `POST /api/progress/completeLesson` - Mark lesson complete

7.4.3 Admin APIs

- (Future enhancement: Admin API endpoints)
-

8. API Documentation

8.1 Courses API

`GET /api/v1/CoursesApi`

Description: Get all available courses

Authentication: Not required

Response:

```
[  
  
  {  
  
    "id": "guid",  
  
    "title": "Python Basics",  
  
    "description": "Learn Python from scratch",  
  
    "difficulty": "Beginner",
```

```
    "language": "Python",

    "lessonCount": 10,

    "createdAt": "2025-01-01T00:00:00Z"

  }

]
```

GET /api/v1/CoursesApi/{id}

Description: Get course by ID

Authentication: Not required

Parameters:

- **id** (Guid) - Course ID

Response:

```
{

  "id": "guid",

  "title": "Python Basics",

  "description": "...",

  "difficulty": "Beginner",

  "language": "Python",

  "lessons": [

    {

      "id": "guid",

      "title": "Introduction",

      "orderNumber": 1
```

```
}  
  
],  
  
  "createdAt": "2025-01-01T00:00:00Z"  
  
}
```

GET /api/v1/CoursesApi/my-courses

Description: Get user's enrolled courses

Authentication: Required

Response: Same as GET /api/v1/CoursesApi with additional `enrolledAt` field

8.2 Code Execution API

POST /api/code/run

Description: Execute code

Authentication: Required

Request:

```
{  
  
  "language": "java",  
  
  "code": "public class Main { ... }",  
  
  "exerciseld": "guid (optional)"  
  
}
```

Response:

```
{  
  
  "output": "Hello World",  
  
  "isCorrect": true,
```

```
"message": "All tests passed",  
  
"testCaseResults": [...],  
  
"passedTests": 5,  
  
"totalTests": 5  
  
}
```

9. Database Schema

9.1 Entity Relationships

ApplicationUser

├── UserCourses (Many-to-Many with Course)

├── LessonCompletions

├── ExerciseSubmissions

├── UserQuizAttempts

├── UserAssessments

├── CVs

└── Projects

Course

├── Lessons (One-to-Many)

├── Quizzes (One-to-Many)

└── UserCourses (Many-to-Many with User)

Lesson

└─ CodingExercises (One-to-Many)

└─ LessonCompletions (One-to-Many)

CodingExercise

└─ ExerciseTestCases (One-to-Many)

└─ ExerciseSubmissions (One-to-Many)

9.2 Key Constraints

- Soft delete pattern (**IsDeleted** flag)
 - GUID primary keys
 - Foreign key relationships
 - Indexes on frequently queried fields
-

10. Security & Authentication

10.1 Authentication Methods

1. **Email/Password**: ASP.NET Core Identity
2. **Google OAuth**: External provider
3. **GitHub OAuth**: External provider

10.2 Authorization

- **Role-based**: Admin vs Regular User
- **Policy-based**: **AdminOnly** policy
- **Claims-based**: **IsAdmin** claim

10.3 Security Features

- Password hashing (Identity framework)
 - HTTPS enforcement
 - CSRF protection (Anti-forgery tokens)
 - XSS protection (Input sanitization)
 - SQL injection prevention (EF Core parameterized queries)
 - Session management with timeout
-

11. Testing & Quality Assurance

11.1 Unit Tests

- **Framework:** xUnit
- **Coverage:** Services, Repositories
- **Mocking:** Moq
- **Database:** InMemory

11.2 Test Structure

CodeWave.Tests/

|— Services/

| |— CodeServiceTests.cs

|— Repositories/

 |— UnitOfWorkTests.cs

11.3 Validation

- **Client-side:** JavaScript validation
 - **Server-side:** ModelState validation
 - **Data annotations:** View model validation
-

12. Deployment & DevOps

12.1 Build Scripts

- **build-safe.ps1** - Safe build script
- **run-app.ps1** - Application startup

12.2 Database Migrations

- EF Core migrations
- Migration scripts for CV, PDF, templates

12.3 Environment Configuration

- `appsettings.json` - Production settings
 - `appsettings.Development.json` - Development settings
 - Connection strings configuration
 - OAuth credentials configuration
-

13. Appendices

13.1 Glossary

- **LMS**: Learning Management System
- **DTO**: Data Transfer Object
- **VM**: ViewModel
- **ORM**: Object-Relational Mapping
- **OAuth**: Open Authorization
- **API**: Application Programming Interface
- **CRUD**: Create, Read, Update, Delete

13.2 References

- ASP.NET Core 9.0 Documentation
- Entity Framework Core Documentation
- MySQL 8.0 Documentation
- Serilog Documentation
- AutoMapper Documentation
- SignalR Documentation
- Chart.js Documentation

13.3 Change Log

Version 2.0 (January 2025)

- Added Serilog logging
- Added AutoMapper
- Added SignalR
- Added Background services
- Added Unit tests
- Added API controllers
- Enhanced validation
- Updated documentation

Version 1.0 (December 2024)

- Initial release
- Core features implemented

Document End























